

Message Passing Programming

6 Hours

Topics covered:

1. Introduction to Message Passing Interface (MPI)
2. Message Passing Model
3. MPI Basic Datatypes and Functions
4. Point-to-point Communication
5. Collective Communication
6. Benchmarking Parallel Performance
7. MPI Error Handling Functions

Introduction to MPI

- The **MPI standard** is the most popular message-passing library interface specification supporting parallel programming
- MPI is a *message-passing parallel programming model*, in which data is moved from the address space of one process to that of another process through cooperative operations on each process
- **MPI is not a programming language**, and all MPI operations are expressed as functions, subroutines, or methods used by C, C++, Fortran-77, and Fortran-95 etc which are part of MPI standard

Message Passing Model

Message-passing model

- The underlying hardware is assumed to be a collection of processors, each with its own local memory
- A processor has direct access only to the instructions and data stored in its local memory
- However, an interconnection network supports message passing between processors
- Processor **A** may send a message containing some of its local data values to processor **B**, giving processor B indirect access to these values
- The existence of the interconnection network provides an implicit communication channel between every pair of processes

Message Passing Model

Message-passing model

- The user specifies the **number of concurrent processes** when the program begins, and the number of active processes **remains constant throughout the execution** of the program
- Every process **executes the same program**, but because each one has a unique ID number, different processes may perform different operations on the same program
- In a message-passing model, processes pass messages both to **communicate** and to **synchronize** with each other

Advantages of message-passing model

- Message-passing programs run well on a wide variety of *MIMD architectures*
- They are a natural fit for *multicomputers*, which *do not support a global address space*
- However, it is also possible to execute message-passing programs on multiprocessors by *using shared variables as message buffers*
- The MPI programs tend to exhibit *high cache hit rates* when executing on multiprocessors, leading to good performance
- *Debugging MPI programs is simpler* than debugging shared-variable programs. Since each process controls its own memory, it is not possible for one process to accidentally overwrite a variable controlled by another process, a common bug in shared-variable programs.

Key concepts of MPI programming

- Used to create *parallel programs*
- Processors communicate *using message passing* via calls to message passing library routines
- Programmers “parallelize” programs by adding *message calls* between *manager process* and *worker process*
- No process can be *created or terminated* in the middle of program execution
- All process *stay alive* till the program terminates
- Each processor has a *local memory* to which it has exclusive access
- The MPI programs tend to exhibit *high cache hit rates* when executing on multiprocessors, leading to good performance
- The number of processes *is fixed* when starting the program

MPI Naming Conventions, Basic Datatypes and Routines

MPI Naming Conventions

- The names of all **MPI entities** (routines, constants, types, etc.) begin with **MPI_** to avoid conflicts

Example: **MPI_Init(&argc, &argv)**

- All **MPI constants** are strings of capital letters and underscores beginning with **MPI_**

Example: **MPI_COMM_WORLD**

Predefined data types for MPI

MPI Datatype

•MPI_CHAR	signed char
MPI_SHORT	signed short int
•MPI_INT	signed int
•MPI_LONG	signed long int
•MPI_LONG_LONG_INT	long long int
•MPI_UNSIGNED_CHAR	unsigned char
•MPI_UNSIGNED_SHORT	unsigned short int
•MPI_UNSIGNED	unsigned int
•MPI_UNSIGNED_LONG	unsigned long int
•MPI_UNSIGNED_LONG_LONG	unsigned long long int
•MPI_FLOAT	float
•MPI_DOUBLE	double
•MPI_LONG_DOUBLE	long double
•MPI_WCHAR	wide char
•MPI_PACKED	special data type for packing
•MPI_BYTE	single byte value

C-Data type

MPI routines

- MPI routines are implemented as **functions** which return the *exit status* of the function call

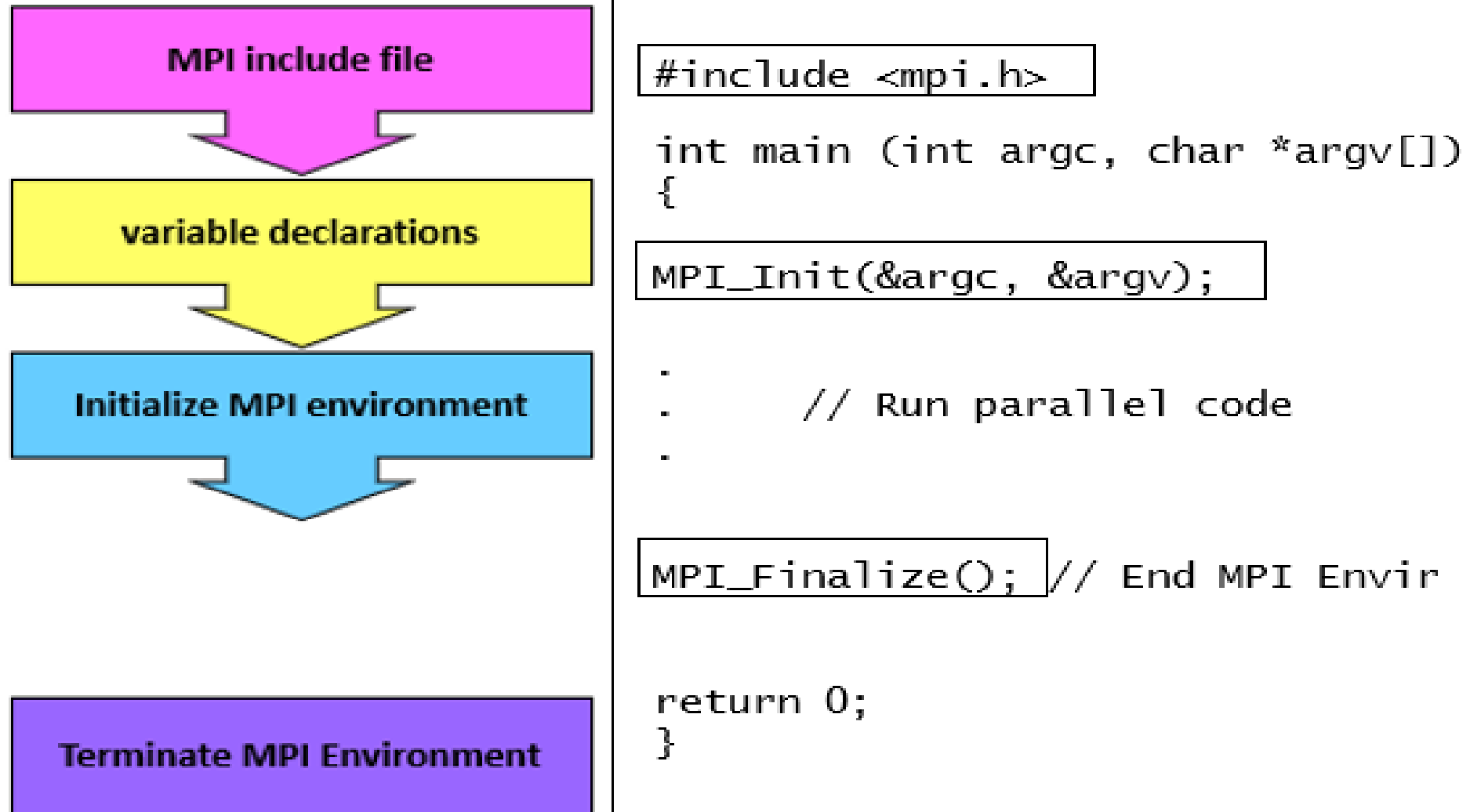
```
int ierr;
```

```
...
```

```
ierr = MPI_Init(&argc, &argv);
```

- The *error code* returned is **MPI_SUCCESS** if the routine ran successfully or else the integer returned has an implementation-dependent value indicating the specific error

General MPI Program Structure



Basic Environment

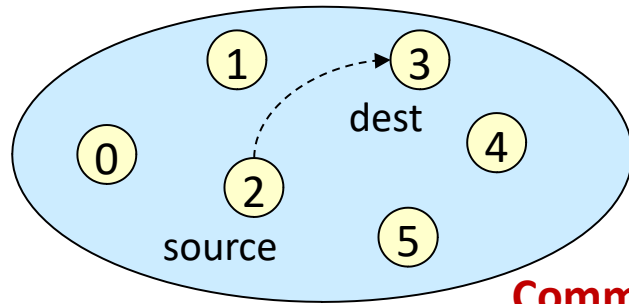
MPI_Init(&argc, &argv)

- Must be called in every MPI program
- It should be the *first MPI function* call made by every MPI process
- It initializes *MPI environment*
- Can be used to pass command line arguments to all

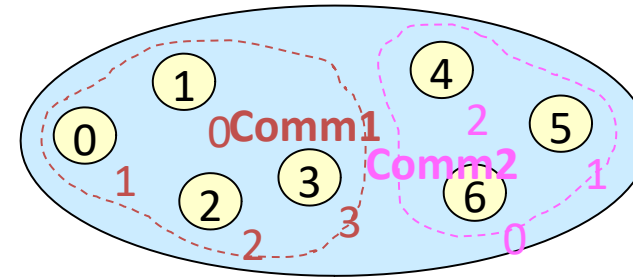
MPI_Finalize()

- It *terminates MPI environment* after releasing all the held up resources
- It should be the *last* MPI function call

Communicators & Rank



Communicator



MPI_COMM_WORLD

MPI_COMM_WORLD

- When MPI has been initialized, every active process becomes a member of a communicator called **MPI_COMM_WORLD**
- A communicator is an object that provides the environment for message passing among processes
- MPI_COMM_WORLD is the *default communicator* that you get "for free"
- However, you can create your own communicators if you need to partition the processes into independent communication groups

Communicators & Rank

What is *rank* of a process??

- Processes within a communicator are *always ordered*
- The rank of a process is *its position* in the overall order
- In a communicator with p processes, each process has a unique rank (ID number) between 0 and $p - 1$
- A process may use its rank to determine which portion of a computation and/or a dataset it is responsible for

Communicators & Rank

```
int my_rank, size;
```

```
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank)
```

- A process calls this function to determine *its rank* within a communicator

```
MPI_Comm_size(MPI_COMM_WORLD, &size)
```

- A process calls this function to determine *the total number of processes* in a communicator

```
int my_rank, size;  
MPI_Init(&argc, &argv);  
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);  
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Hello World for MPI

```
#include <mpi.h>
#include<stdio.h>

int main (int argc, char *argv[])

{   int rank, size;

    MPI_Init (&argc, &argv);           //initialize MPI
    MPI_Comm_size(MPI_COMM_WORLD, &size);    //get number of processes
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);    //get my process id

    printf("Processor %d of %d: Hello World!\n", rank, size);

    MPI_Finalize();                     //MPI cleanup

    return 0;
}
```


Hello World for MPI

- Running this code on four processors will produce a result like:

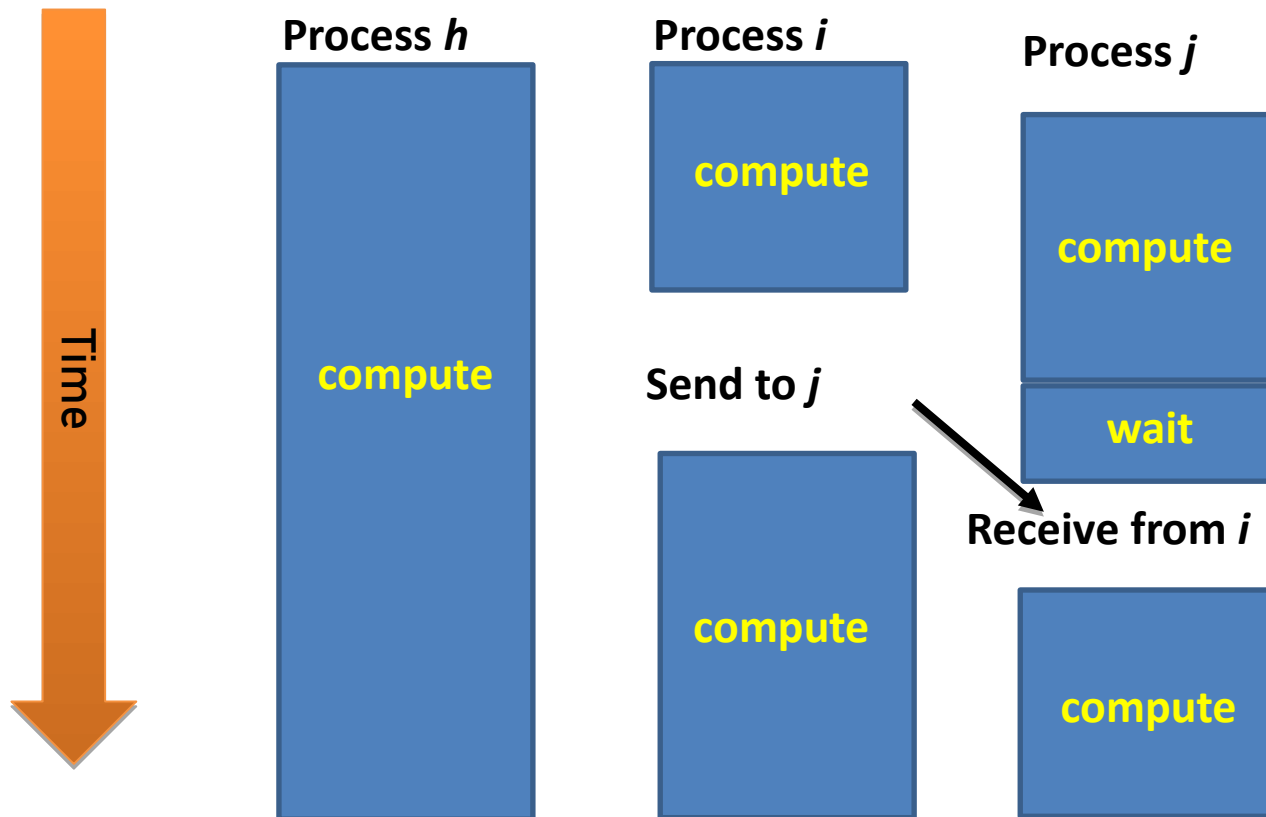
```
mpicc -o prg1 program1.c  
mpirun -np 4 ./prg1
```

```
Processor 2 of 4: Hello World!  
Processor 1 of 4: Hello World!  
Processor 3 of 4: Hello World!  
Processor 0 of 4: Hello World!
```

- Each processor executes the same code, including probing for its rank and size and printing the string.
- The order of the printed lines is essentially random!

Point-to-point Communication in MPI

- A point-to-point communication involves a *pair of processes*
- In the following example, process *h* is not involved in a communication. It continues executing statement, manipulating its local variables. Process *i* performs local computations, then sends a message to process *j*. After the message is sent, it continues on with its computation. Process *j* performs local computations, then blocks until it receives a message from process *i*.



MPI Errors

MPI_SUCCESS → No error; MPI routine completed successfully

MPI_ERR_COMM → Invalid communicator

MPI_ERR_TYPE → Invalid datatype argument

MPI_ERR_COUNT → Invalid count argument

MPI_ERR_TAG → Invalid tag argument

MPI_ERR_RANK → Invalid source or destination rank

Blocking Message Passing Routines

MPI_Send

```
MPI_Send(void *message,    //address of send buffer
         int  count,       //number of data items in send buffer
         MPI_Datatype datatype, // type of data to be transmitted
         int  dest,       // rank of the process to receive the data
         int  tag,        // integer label for the message
         MPI_Comm comm    // communicator )
```

- This routine sends a message and *block* until the application buffer in the sending task is *free* for reuse
- The MPI implementation may buffer your send allowing it to return almost immediately
- If the implementation *does not buffer the send*, the send will not complete until the matching receive occurs

Blocking Message Passing Routines

MPI_Recv

MPI_ANY_SOURCE

MPI_ANY_TAG

```
MPI_Recv(void *message,    //address of receive buffer
         int  count,       //maximum number of data items to be received
         MPI_Datatype datatype, // type of data to be received
         int  source,      // rank of the process sending the data
         int  tag,         // integer label for the message
         MPI_Comm comm     // communicator
         MPI_Status *status // status information of data received )
```

- This routine returns *only after the requested data is available* in the application buffer
- The *status record* contains information about the *just-completed receive function*. In particular:

```
struct MPI_Struct {
    int MPI_SOURCE;
    int MPI_TAG;
    int MPI_ERROR;
    int _cancelled;
    size_t _ucount;
};
```

Blocking send-recv (2-processes)

```
#include <mpi.h> #include <stdio.h>

int main (int argc, char *argv[]) {

    int rank, size, send_number;

    MPI_Init (&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank == 0){
        send_number = 777;
        MPI_Send(&send_number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    else if (rank == 1) {
        MPI_Recv(&send_number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1 received number %d from process 0 \n", send_number);
    }

    MPI_Finalize();

    return 0;
}
```

Blocking send-recv (n-processes)

```
#include <mpi.h> #include <stdio.h>

int main (int argc, char *argv[]) {

    int rank, size, send_number;

    MPI_Init (&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank == 0){
        send_number = 777;
        MPI_Send(&send_number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    else if (rank == 1) {
        MPI_Recv(&send_number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1 received number %d from process 0 \n", send_number);
    }
    else
        printf("Hello from process %d \n", rank);

    MPI_Finalize();

    return 0;
}
```

Blocking send-recv (using MPI_Status -1)

```
#include <mpi.h> #include <stdio.h>

int main (int argc, char *argv[]) {

    int rank, size, send_number;

    MPI_Status status;

    MPI_Init (&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank == 0){
        send_number = 999;
        MPI_Send(&send_number, 1, MPI_INT, 1, 3, MPI_COMM_WORLD);
    }
    if (rank == 1) {
        MPI_Recv(&send_number, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
        printf("Process %d received number %d from process %d in message %d \n", rank, send_number,
            status.MPI_SOURCE, status.MPI_TAG);
    }
    else
        printf("Hello from process %d \n", rank);

    MPI_Finalize();

    return 0;
}
```


Blocking send-recv (using MPI_Status -2)

```
#include <mpi.h> #include <stdio.h>

int main (int argc, char *argv[]) {

    int rank, size, send_number0, send_number1, recv_buffer;

    MPI_Status status;

    MPI_Init (&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank == 0){
        send_number0 = 555;
        MPI_Send(&send_number0, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    if(rank == 1){
        send_number1 = 777;
        MPI_Send(&send_number1, 1, MPI_INT, 1, 1, MPI_COMM_WORLD);
    }
    while(1) {
        MPI_Recv(&recv_buffer, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
        printf("Process %d received number %d from process %d in message %d \n", rank, send_number,
            status.MPI_SOURCE, status.MPI_TAG);
    }
    MPI_Finalize();
    return 0;
}
```

Synchronous Message Passing Routines

MPI_Ssend

```
MPI_Ssend(void *message,    //address of data to be transmitted
          int  count,        //number of data items
          MPI_Datatype datatype, // type of data to be transmitted
          int  dest,         // rank of the process to receive the data
          int  tag,          // integer label for the message
          MPI_Comm comm      // communicator )
```

- This routing sends a message and *block* until the application buffer in the sending task is *free* for reuse **and the destination process has started to receive the message**

Synchronous Ssend-recv (n-processes)

```
#include <mpi.h> #include <stdio.h>

int main (int argc, char *argv[]) {

    int rank, size, send_number;

    MPI_Init (&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank == 0){
        send_number = 555;
        MPI_Ssend(&send_number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    else if (rank == 1) {
        MPI_Recv(&send_number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1 received number %d from process 0 \n", send_number);
    }
    else
        printf("Hello from process %d \n", rank);

    MPI_Finalize();

    return 0;
}
```

Buffered Message Passing Routines

MPI_Bsend

```
MPI_Bsend(void *message,    //address of data to be transmitted
          int  count,        //number of data items
          MPI_Datatype datatype, // type of data to be transmitted
          int  dest,         // rank of the process to receive the data
          int  tag,          // integer label for the message
          MPI_Comm comm      // communicator )
```

- This routine permits the programmer to *allocate the required amount of buffer space* into which data can be copied until it is delivered
- Insulates against the problems associated with *insufficient system buffer space*
- Routine returns after the data has been copied from application buffer space to the allocated buffer
- It must be used with the **MPI_Buffer_attach()** and **MPI_Buffer_detach()** routines

Buffered Message Passing Routines

MPI_Buffer_attach

```
MPI_Buffer_attach (void *buffer, // address of the buffer  
                  int size      // buffer size in bytes )
```

MPI_Buffer_detach

```
MPI_Buffer_detach (void *buffer, // address of the buffer  
                  int *size      // buffer size in bytes )
```

- Used by programmer to allocate/deallocate message buffer space to be used by the **MPI_Bsend()** routine
- **The *size* argument is specified in actual data bytes** - not a count of data elements
- *Only one buffer* can be attached to a process at a time

Buffered Bsend-recv (2-processes)

```
#include <mpi.h> #include<stdio.h> #include <stdlib.h>

int main (int argc, char *argv[]) {

int rank, size, data_send=12345, received ;

int buffer_attached_size = MPI_BSEND_OVERHEAD + sizeof(int);
char* buffer_attached = (char*)malloc(buffer_attached_size);

MPI_Init (&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

if(rank == 0){
    MPI_Buffer_attach(buffer_attached, buffer_attached_size);
    printf("Sender %d sent value %d \n", rank, data_send);
    MPI_Bsend(&data_send, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    MPI_Buffer_detach(&buffer_attached, &buffer_attached_size);
    free(buffer_attached);
}
else if (rank == 1) {
    MPI_Recv(&received, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf(" Receiver %d received value: %d \n", rank, received);
}
MPI_Finalize();
return 0;
}
```

Broadcasting using send-recv

```
#include <mpi.h>
#include<stdio.h>
int main (int argc, char *argv[]) {
    int i, root_rank = 0, my_rank, size, recv_buffer;

    MPI_Init (&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    if(my_rank == 0){
        int send_buffer=777;
        for(i=1; i<size; i++)
            MPI_Send(&send_buffer, 1, MPI_INT, i, 2, MPI_COMM_WORLD);
    }
    else{
        MPI_Recv(&recv_buffer, 1, MPI_INT, 0, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received number %d from process 0 \n", my_rank, recv_buffer);
    }
    MPI_Finalize();
    return 0;
}
```

Deadlock

"A process is in a deadlock stale if it is blocked waiting for a condition that will never become true"

Deadlock (Recv-Recv)

```
int a,b,c;
int rank;
MPI_Status status;
.....
if(rank==0)
{
    MPI_Recv(&b,1,MPI_INT,1,0,MPI_COMM_WORLD,&status);
    MPI_Send(&a,1,MPI_INT,1,0,MPI_COMM_WORLD);
    c=a+b/2;
}
else if(rank==1)
{
    MPI_Recv(&a,1,MPI_INT,0,0,MPI_COMM_WORLD,&status);
    MPI_Send(&b,1,MPI_INT,0,0,MPI_COMM_WORLD);
    c=a+b/2;
}
```

Deadlock (Tag mismatch)

```
int a,b,c;
int rank;
MPI_Status status;
.....
if(rank==0)
{
    MPI_Send(&a,1,MPI_INT,1,1,MPI_COMM_WORLD);
    MPI_Recv(&b,1,MPI_INT,1,1,MPI_COMM_WORLD,&status);
    c=a+b/2;
}
else if(rank==1)
{
    MPI_Send(&b,1,MPI_INT,0,0,MPI_COMM_WORLD);
    MPI_Recv(&a,1,MPI_INT,0,0,MPI_COMM_WORLD,&status);
    c=a+b/2;
}
```

Deadlock (Rank mismatch)

```
int a,b,c;
int rank;
MPI_Status status;
.....
if(rank==0)
{
    MPI_Send(&a,1,MPI_INT,2,1,MPI_COMM_WORLD);
    MPI_Recv(&b,1,MPI_INT,2,1,MPI_COMM_WORLD,&status);
    c=a+b/2;
}
else if(rank==1)
{
    MPI_Send(&b,1,MPI_INT,0,0,MPI_COMM_WORLD);
    MPI_Recv(&a,1,MPI_INT,0,0,MPI_COMM_WORLD,&status);
    c=a+b/2;
}
```

Deadlock (Communicator mismatch)

```
int a,b,c;
int rank;
MPI_Status status;
.....
if(rank==0)
{
    MPI_Send(&a,1,MPI_INT,1,1,My_Communicator);
    MPI_Recv(&b,1,MPI_INT,1,1,MPI_COMM_WORLD,&status);
    c=a+b/2;
}
else if(rank==1)
{
    MPI_Send(&b,1,MPI_INT,0,0,MPI_COMM_WORLD);
    MPI_Recv(&a,1,MPI_INT,0,0,MPI_COMM_WORLD,&status);
    c=a+b/2;
}
```

Deadlock (self blocking Send)

```
int a,b,c;
int rank;
MPI_Status status;
.....
if(rank==0)
{
    MPI_Send(&a,1,MPI_INT,0,1,MPI_COMM_WORLD);
    MPI_Recv(&b,1,MPI_INT,1,1,MPI_COMM_WORLD,&status);
    c=a+b/2;
}
else if(rank==1)
{
    MPI_Send(&b,1,MPI_INT,0,0,MPI_COMM_WORLD);
    MPI_Recv(&a,1,MPI_INT,0,0,MPI_COMM_WORLD,&status);
    c=a+b/2;
}
```

Collective Communication in MPI

- A collective communication is a communication operation in which *a group of processes works together* to **distribute** or **gather** together a set of one or more values
- **Scope:**
 - Collective communication routines must involve **all** processes within the scope of a communicator
 - All processes are by default, members in the communicator MPI_COMM_WORLD
 - Unexpected behavior, including program failure, can occur if even one process in the communicator doesn't participate
 - It is the programmer's responsibility to ensure that all processes within a communicator participate in any collective operations.

Types of Collective Operations

1. Synchronization:

processes wait until all members of the group have reached the synchronization point

2. Data Movement:

processes send/receive data among themselves

3. Collective Computation:

one or more member of the group collects data from the other members and performs an operation (min, max, add, multiply, etc.) on that data

Predefined MPI reduction operators

<u>Operator</u>	<u>Meaning</u>
MPI_BAND	Bitwise and
MPI_BOR	Bitwise or
MPI_BXOR	Bitwise exclusive or
MPI_LAND	Logical and
MPI_LOR	Logical or
MPI_LXOR	Logical exclusive or
MPI_MAX	Maximum
MPI_MAXLOC	Maximum and location of maximum
MPI_MIN	Minimum
MPI_MINLOC	Minimum and location of minimum
MPI_PROD	Product
MPI_SUM	Sum

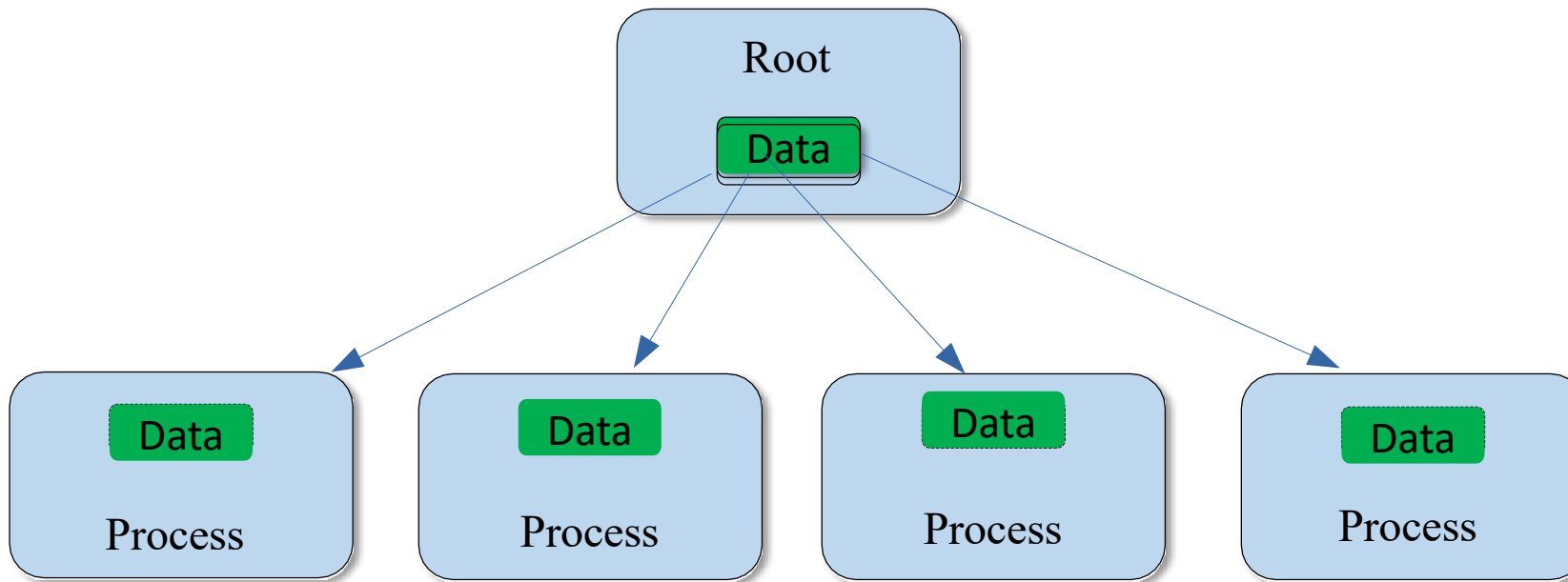
Collective Communication Routines

- MPI_Bcast()** → Broadcast data from root to all other processes
- MPI_Alltoall()** → Sends/Receives data from every processes to all processes in chunks
- MPI_Reduce()** → Combine values from all processes to a single value
- MPI_Scatter()** → Scatters buffer in parts to group of processes
- MPI_Gather()** → Gather values from group of processes
- MPI_Allgather()** → Every process gather values from all processes in a communicator
- MPI_Scan()** → Computes the scan (partial reductions) of data on a collection of processes

MPI_Bcast (one-to-all)

```
MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm)
```

- Broadcasts a message from process with rank **root** to all other processes in the same *communicator*.
- Must be called by all the processes *with the same arguments Data*



buffer  starting address of buffer

count  number of entries in buffer (integer)

datatype  data type of buffer

root  rank of broadcast root (integer)

comm  communicator (handle)

MPI_Bcast

Broadcasts a message to all other processes of that group

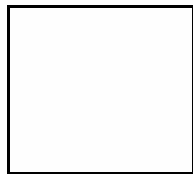
count = 1;

source = 1;

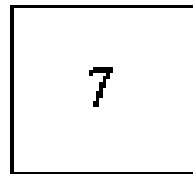
broadcast originates in task 1

MPI_Bcast(&msg, count, MPI_INT, source, MPI_COMM_WORLD);

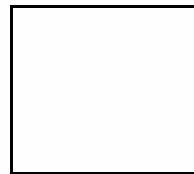
task 0



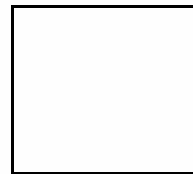
task 1



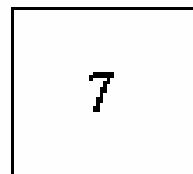
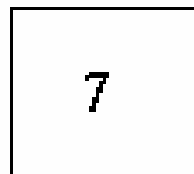
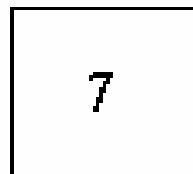
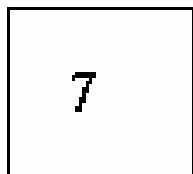
task 2



task 3



← msg (before)



← msg (after)

MPI_Bcast

```
#include <mpi.h>
#include <stdio.h>
int main (int argc, char *argv[]) {
    int rank;
    int buf;
    const int root=0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank == root){
        printf("I am Root: enter a number to broadcast \n");
        scanf("%d",&buf);
    }

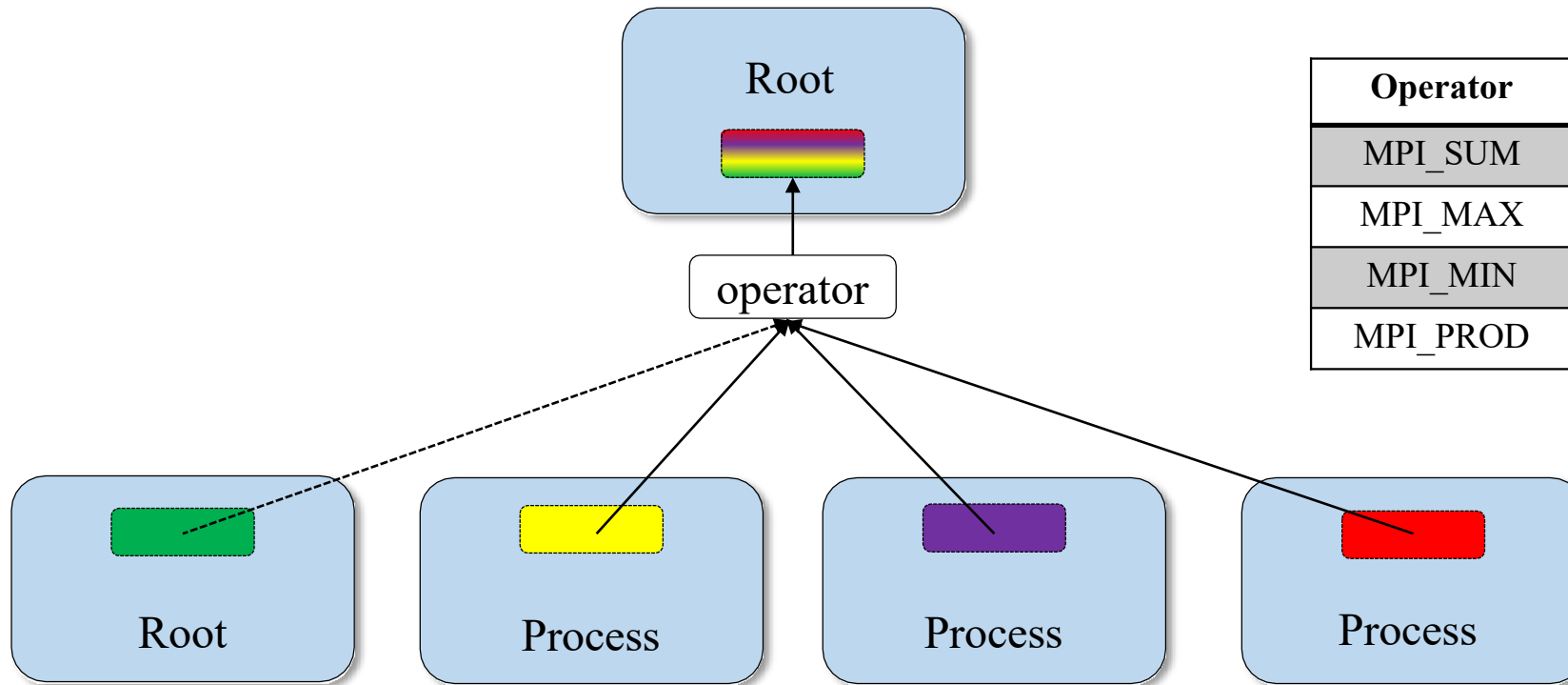
    /* everyone calls bcast, data is taken from root and ends up in everyone's buf */
    MPI_Bcast(&buf, 1, MPI_INT, root, MPI_COMM_WORLD);
    printf("Process[%d]: After Bcast, buf is %d\n", rank, buf);

    MPI_Finalize();
    return 0;
}
```

MPI_Reduce

```
MPI_Reduce(void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype,  
            MPI_Op op, int root, MPI_Comm comm)
```

- One process (root) collects data from all the other processes in the same communicator, and performs an operation on the data (i.e combines elements provided by input buffer of each process in the group using operation *op*.)
- Returns combined value in the output buffer of process with rank *root*



Operator
MPI_SUM
MPI_MAX
MPI_MIN
MPI_PROD

sendbuf  address of send buffer

recvbuf  address of result buffer

count  number of elements in send buffer (integer)

datatype  data type of send buffer

op  reduction operation

Root : rank of broadcast root (integer)

MPI_Reduce

Perform and associate reduction operation across all tasks in the group and place the result in one task

```
count = 1;
```

```
dest = 1;
```

result will be placed in task 1

```
MPI_Reduce(sendbuf, recvbuf, count, MPI_INT, MPI_SUM,  
           dest, MPI_COMM_WORLD);
```

task 0

1

task 1

2

task 2

3

task 3

4

← sendbuf (before)

10

← recvbuf (after)

MPI_Reduce – sending process rank

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
int main (int argc, char *argv[]) {
    int root_rank = 0, size, my_rank;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(size != 4)
    {
        printf("This application is meant to be run with 4 MPI processes.\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }

    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    // Each MPI process sends its rank to reduction, root MPI process collects the result
    int reduction_result = 0;
    printf("Sending my rank from process %d \n", my_rank);
    MPI_Reduce(&my_rank, &reduction_result, 1, MPI_INT, MPI_SUM, root_rank, MPI_COMM_WORLD);

    if(my_rank == root_rank)
    {
        printf("The sum of all ranks is %d.\n", reduction_result);
    }

    MPI_Finalize();
    return 0;
}
```

MPI_Reduce – computing partial result

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
int main (int argc, char *argv[]) {
    int root_rank = 0, size, my_rank, sum=0, reduction_result=0;
    int arr[4]={1,2,3,4};
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(size != 3)
    {
        printf("This application is meant to be run with 3 MPI processes.\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }

    // Find partial sum
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    sum = sum + arr[my_rank]+arr[my_rank+1];
    printf("My rank is %d and the partial sum is %d \n", my_rank, sum);

    // Each MPI process sends its rank to reduction, root MPI process collects the result
    MPI_Reduce(&sum, &reduction_result, 1, MPI_INT, MPI_SUM, root_rank, MPI_COMM_WORLD);

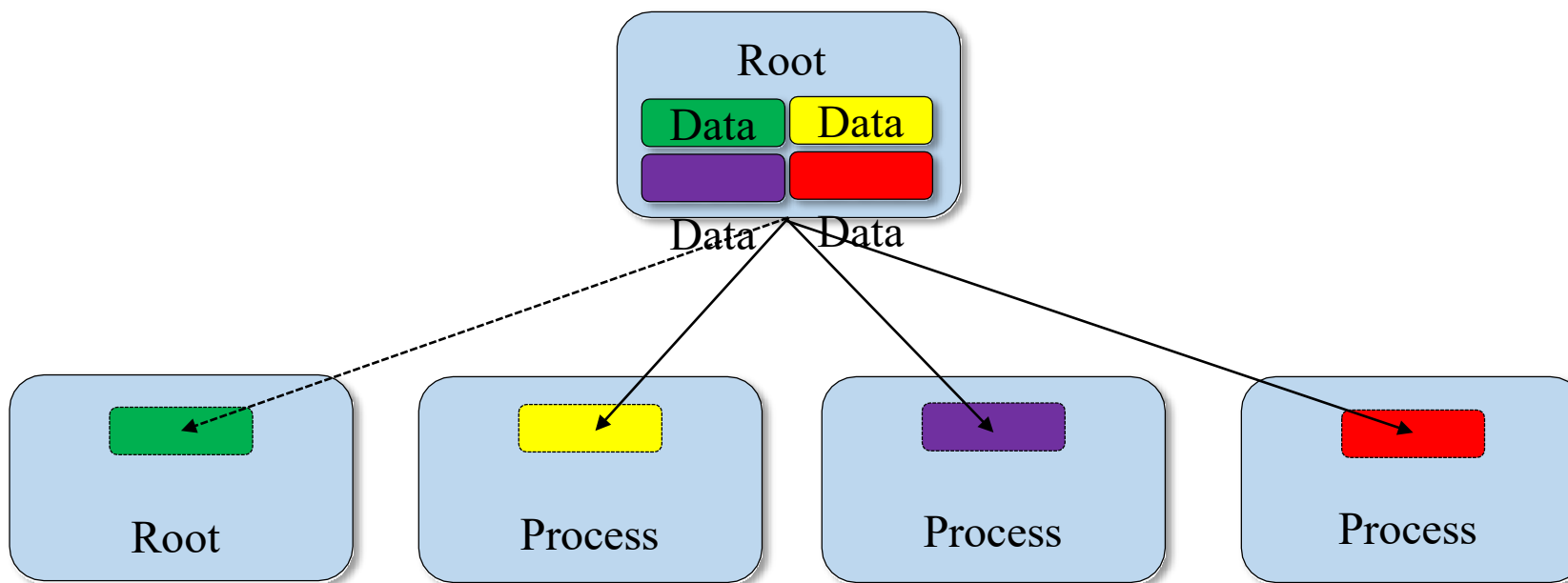
    if(my_rank == root_rank)
    {
        printf("The sum of all ranks is %d.\n", reduction_result);
    }

    MPI_Finalize();
    return 0;
}
```


MPI_Scatter

```
MPI_Scatter (void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf,  
             int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)
```

- Sends individual messages from the root process to all other processes
- Inverse to MPI_Gather
- *sendbuf* is ignored by all non-root processes



sendbuf 🖨 address of send buffer (significant only at root)

sendcount : number of elements sent to each process (significant only at root)

sendtype 🖨 data type of send buffer elements (significant only at root)

recvcount 🖨 number of elements in receive buffer (integer)

recvtype : data type of receive buffer elements

root 🖨 rank of sending process (integer)

MPI_Scatter

Sends data from one task to all other tasks in a group

```
sendcnt = 1;
```

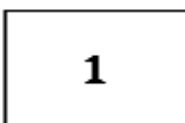
```
recvcnt = 1;
```

```
src = 1;
```

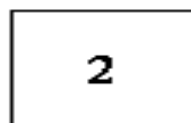
task 1 contains the message to be scattered

```
MPI_Scatter(sendbuf, sendcnt, MPI_INT,  
            recvbuf, recvcnt, MPI_INT,  
            src, MPI_COMM_WORLD);
```

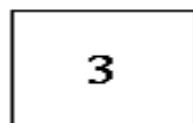
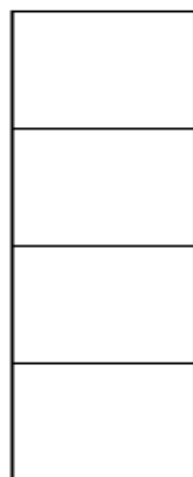
task 0



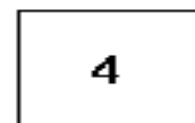
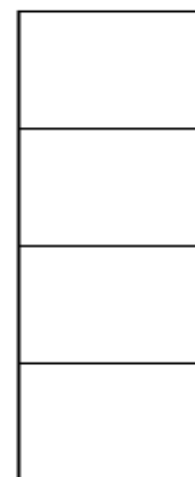
task 1



task 2



task 3



← sendbuf (before)

← recvbuf (after)

MPI_Scatter – sending single data

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[]) {
    int root_rank = 0, size, my_rank, recv_buf;
    int send_buf[4]={10,20,30,40};
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(size != 4)
    {
        printf("This application is meant to be run with 4 MPI processes.\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    // Root process will scatter the data to other processes including itself
    MPI_Scatter(send_buf, 1, MPI_INT, &recv_buf, 1, MPI_INT, root_rank, MPI_COMM_WORLD);
    printf("data in process[%d] = %d", my_rank, recv_buf);

    MPI_Finalize();
    return 0;
}
```

MPI_Scatter – sending multiple data

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[]) {
    int root_rank = 0, size, my_rank, i;
    int send_buf[4]={10,20,30,40};
    int recv_buf[2];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(size != 2)
    {
        printf("This application is meant to be run with 2 MPI processes.\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    // Root process will scatter the data to other processes including itself
    MPI_Scatter(send_buf, 2, MPI_INT, &recv_buf, 2, MPI_INT, root_rank, MPI_COMM_WORLD);

    printf("Data in process [%d] is: \n", my_rank);
    for( i=0; i<2; i++)
        printf("%d \n", recv_buf[i]);

    MPI_Finalize();
    return 0;
}
```

MPI_Scatter – sending string

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main (int argc, char *argv[]) {
    int root_rank = 0, size, my_rank;
    char *send_buf="HELLO WORLD";
    int len = strlen(send_buf);
    char *recv_buf = (char *)malloc(len*sizeof(char));

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(size != 6)
    {
        printf("This application is meant to be run with 6 MPI processes.\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    // Root process will scatter the data to other processes including itself
    MPI_Scatter(send_buf, 2, MPI_CHAR, recv_buf, 2, MPI_CHAR, root_rank, MPI_COMM_WORLD);

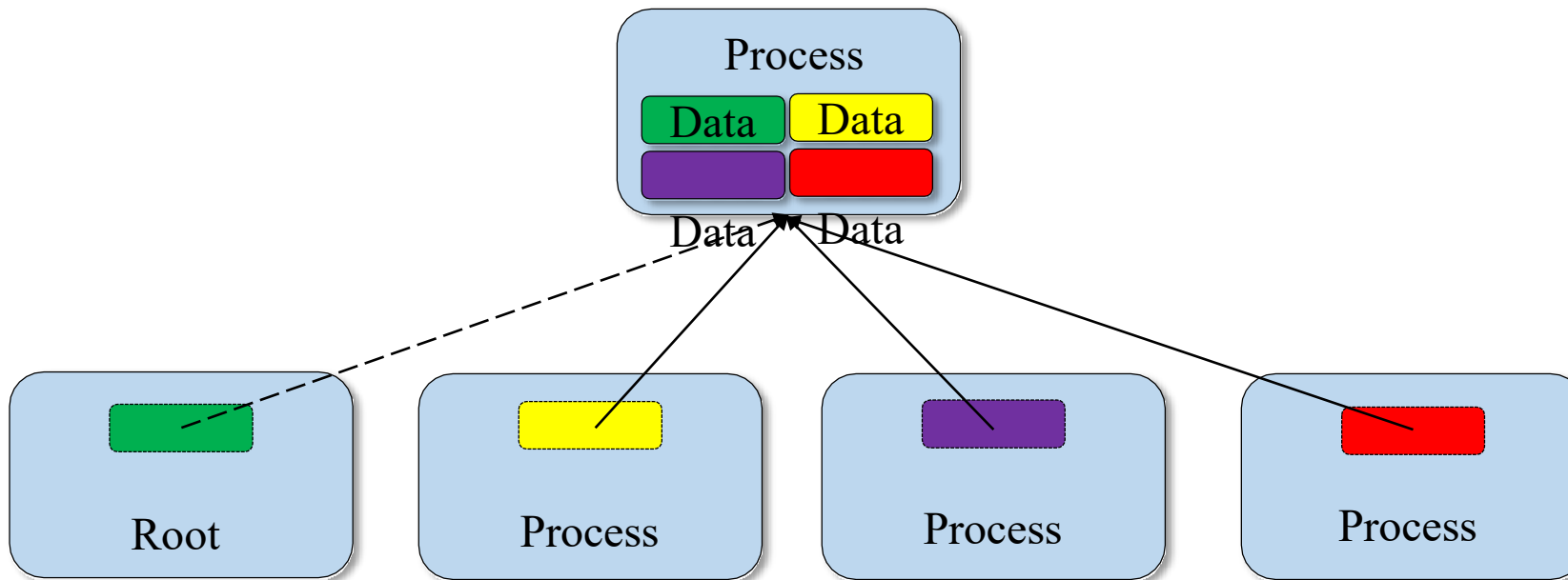
    printf("Data in process [%d] is: %s \n", my_rank, recv_buf);

    MPI_Finalize();
    return 0;
}
```

MPI_Gather

```
MPI_Gather (void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf,  
            int recvcnt, MPI_Datatype recvtype, int root, MPI_Comm comm)
```

- One process (*root*) collects data from all the other processes in the same communicator (i.e each process in *comm* (including *root* itself) sends its *sendbuf* to *root*.)
- The *root* process receives the messages in *recvbuf* **in rank order**
- Must be called by all the processes with the same arguments



Inverse to MPI_Scatter

MPI_Gather

Gathers together values from a group of processes

```
sendcnt = 1;
```

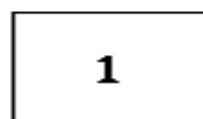
```
recvcnt = 1;
```

```
src = 1;
```

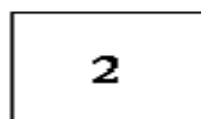
messages will be gathered in task 1

```
MPI_Gather(sendbuf, sendcnt, MPI_INT,  
recvbuf, recvcnt, MPI_INT,  
src, MPI_COMM_WORLD);
```

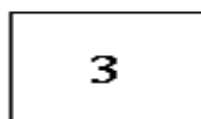
task 0



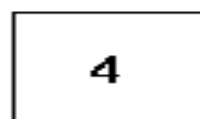
task 1



task 2



task 3



← sendbuf (before)



← recvbuf (after)

MPI_Gather – sending numbers

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
int main (int argc, char *argv[]) {
    int root_rank = 0, size, my_rank;
    int send_buf = 111;
    int recv_buf[4];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(size != 4)
    {
        printf("This application is meant to be run with 4 MPI processes.\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }

    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    send_buf = send_buf + my_rank;

    // Each MPI process sends its send_buf to Root process which collects the data in rank order
    printf("Sending data %d from process %d\n", send_buf, my_rank);
    MPI_Gather(&send_buf, 1, MPI_INT, &recv_buf, 1, MPI_INT, root_rank, MPI_COMM_WORLD);

    // Display result in Root
    if(my_rank == root_rank)
    {
        printf("The gathered data at Root is\n");
        for(int i=0; i<4; i++)
            printf("%d\n", recv_buf[i]);
    }

    MPI_Finalize();
    return 0;
}
```


MPI_Gather – sending Strings

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main (int argc, char *argv[]) {
    int root_rank = 0, size, my_rank;
    char send_buf[10] = "AA ";
    char *recv_buf;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    int len = strlen(send_buf);
    send_buf[len-2] = send_buf[len-2] + my_rank;
    recv_buf = (char *)malloc(size*len*sizeof(char));

    // Each MPI process sends its send_buf to Root process which collects the data in rank order
    printf("Sending data %s from process %d\n", send_buf, my_rank);
    MPI_Gather(send_buf, len, MPI_CHAR, recv_buf, len, MPI_CHAR, root_rank, MPI_COMM_WORLD);

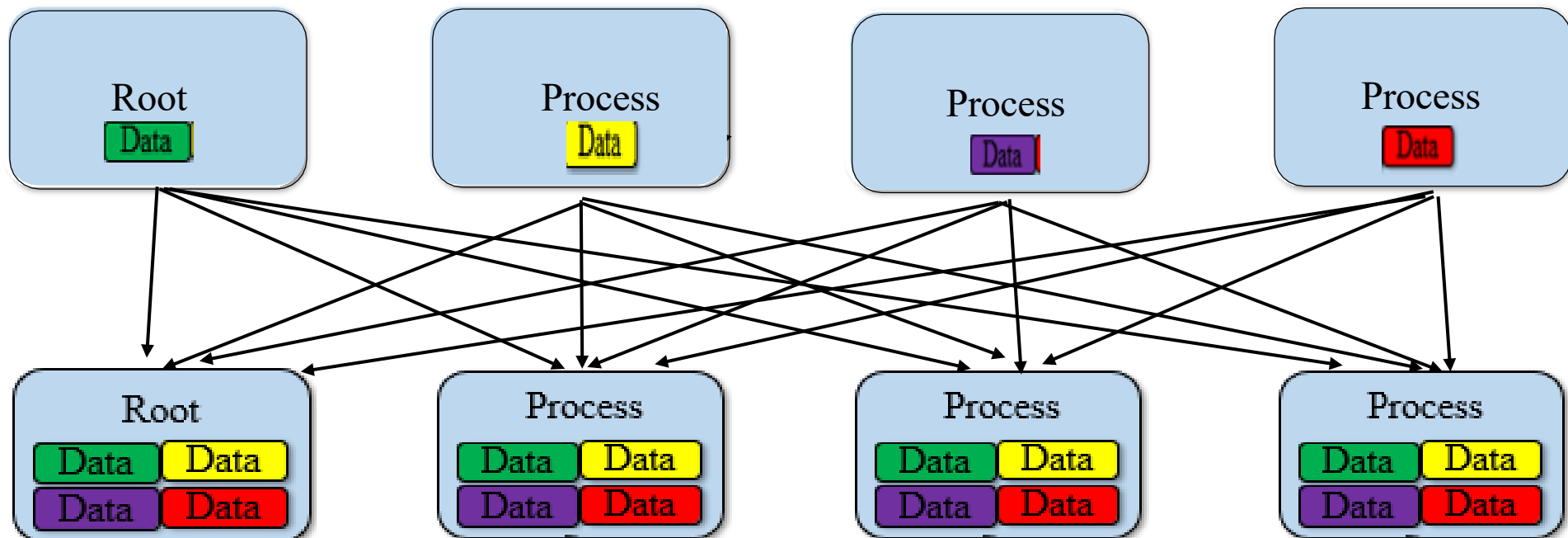
    // Display result in Root
    if(my_rank == root_rank)
        printf("The gathered data at Root is : %s\n", recv_buf);

    MPI_Finalize();
    return 0;
}
```

MPI_Allgather

```
MPI_Allgather (void *sendbuf, int sendcnt, MPI_Datatype sendtype, void  
                *recvbuf, int recvcnt, MPI_Datatype recvtype, MPI_Comm comm)
```

- All the processes collect data from all the other processes in the same communicator (i.e. similar to MPI_Gather except now all processes receive the result.)
- *recvbuf* is **NOT** ignored
- Must be called by all the processes with the same arguments



MPI_Allgather – sending Strings

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main (int argc, char *argv[]) {
    int size, my_rank;
    char send_buf[10] = "AA ";
    char *recv_buf;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    int len = strlen(send_buf);
    send_buf[len-2] = send_buf[len-2] + my_rank;
    recv_buf = (char *)malloc(size*len*sizeof(char));

    // Each MPI process sends its send_buf to Root process which collects the data in rank order
    printf("Sending data %s from process %d\n", send_buf, my_rank);
    MPI_Allgather(send_buf, len, MPI_CHAR, recv_buf, len, MPI_CHAR, MPI_COMM_WORLD);

    // Display gathered result
    printf("The gathered data at Process [%d] is : %s\n", my_rank, recv_buf);

    MPI_Finalize();
    return 0;
}
```

MPI_Alltoall

```
MPI_Alltoall (void *sendbuf, int sendcount, MPI_Datatype sendtype, void  
              *recvbuf, int recvcount, MPI_Datatype recvtyp, MPI_Comm comm )
```

- It is a combination of **MPI_Scatter** and **MPI_Gather**
- It is an extension of the **MPI_Allgather** function
- Each process sends distinct data to each of the receivers. **The j^{th} block that is sent from process i is received by process j and is placed in the i^{th} block of the receive buffer**

Input Data					MPI_Alltoall Result				
P0	0	1	2	3	P0	0	4	8	12
P1	4	5	6	7	P1	1	5	9	13
P2	8	9	10	11	P2	2	6	10	14
P3	12	13	14	15	P3	3	7	11	15

MPI_Allgather vs *MPI_Alltoall*

rank	send buf		recv buf
----	-----		-----
0	a,b,c	MPI_Allgather	a,b,c,A,B,C,#,@,%
1	A,B,C	----->	a,b,c,A,B,C,#,@,%
2	#, @, %		a,b,c,A,B,C,#,@,%

This is just the regular `MPI_Gather`, only in this case all processes receive the data chunks, i.e. the operation is root-less.

rank	send buf		recv buf
----	-----		-----
0	a,b,c	MPI_Alltoall	a,A,#
1	A,B,C	----->	b,B,@
2	#, @, %		c,C,%

(a more elaborate case with two elements per process)

rank	send buf		recv buf
----	-----		-----
0	a,b,c,d,e,f	MPI_Alltoall	a,b,A,B,#,@
1	A,B,C,D,E,F	----->	c,d,C,D,%,\$
2	#, @, %, \$, &, *		e,f,E,F,&,*

MPI_Alltoall

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[]) {
    int size, my_rank,i;
    int send_buf[4] = {10,20,30,40};
    int recv_buf[4];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    if(size != 4)
    {
        printf("This application is meant to be run with 4 MPI processes.\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }

    for(i=0; i<4;i++)
        send_buf[i] += my_rank;

    // Each MPI process sends/recives data to other processes in chunks
    printf("Sending [--%d--%d--%d--%d--] from process %d \n", send_buf[0],send_buf[1],send_buf[2],send_buf[3],my_rank);
    MPI_Alltoall(send_buf, 1, MPI_INT, recv_buf, 1, MPI_INT, MPI_COMM_WORLD);

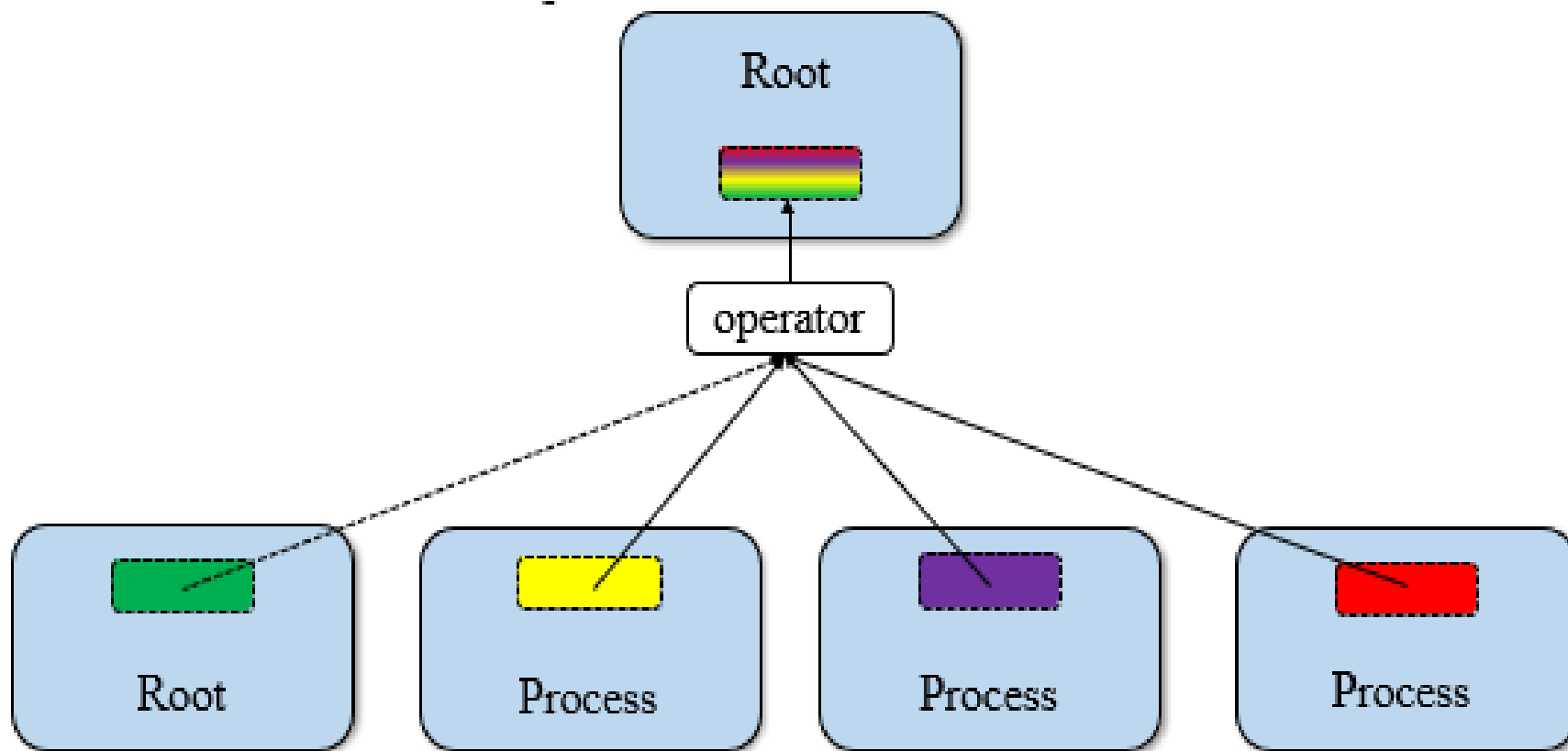
    // Display gathered result
    printf("\n Data gathered in process %d  is: [--%d--%d--%d--%d--] \n", my_rank, recv_buf[0],recv_buf[1],
                                                recv_buf[2],recv_buf[3]);

    MPI_Finalize();
    return 0;
}
```

MPI_Scan

```
MPI_Scan (void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype,  
          MPI_Op op, MPI_Comm comm )
```

- It returns the *partial operation results* on **each processor**



MPI_Reduce vs *MPI_Scan*

- A **reduction** means all processes get the same value while **scan** returns the partial operation results on each process
- **For example:**
 - if you had **10** processes and you were taking the sum of their rank, **MPI_Reduce** would give you the scalar **45** ($0+1+2+3+4+5+6+7+8+9$) on the root process,
 - while **MPI_scan** would give you the scalar result of the reduction *up to the rank of that process on each process*. So processor **0** would get **0**, processor **1** would get **1**, processor **2** would get **3**, and so on. Processor **9** would get **45**

MPI_Scan

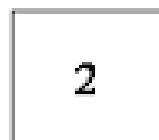
Computes the scan (partial reductions) of data
on a collection of processes

```
count = 1;  
MPI_Scan(sendbuf, recvbuf, count, MPI_INT, MPI_SUM,  
        MPI_COMM_WORLD);
```

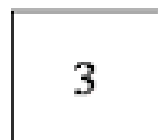
task 0



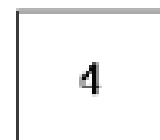
task 1



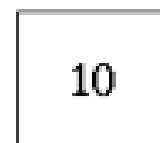
task 2



task 3



← sendbuf (before)



← recvbuf (after)

```
#include <mpi.h>
#include <stdio.h>
```

```
int main (int argc, char *argv[]) {
    int root_rank = 0, size, my_rank, result=0;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

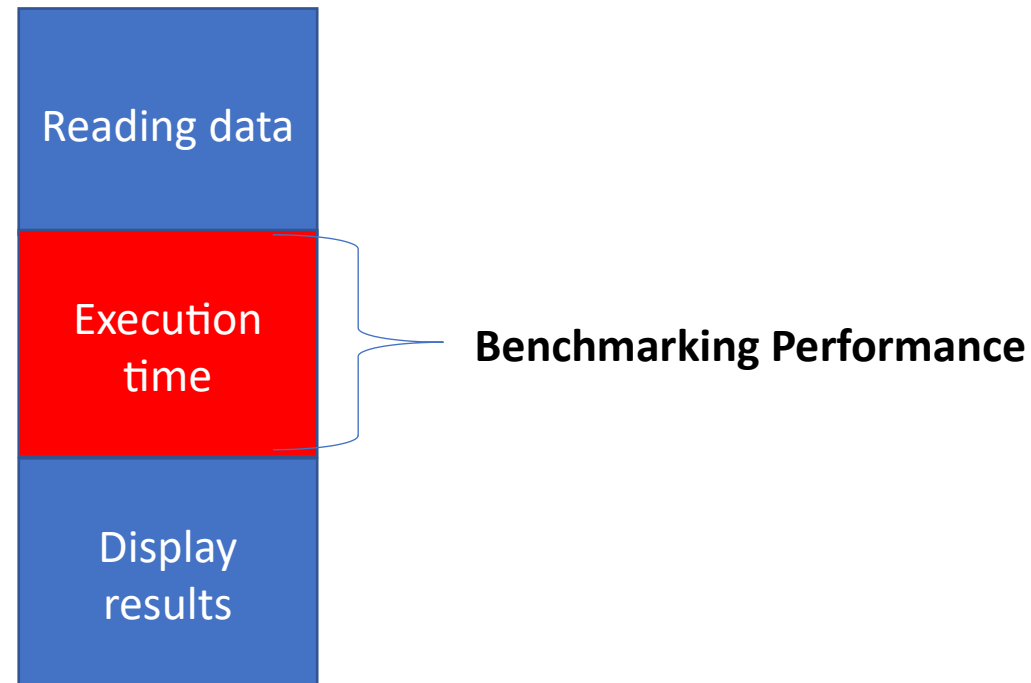
    // Each MPI process sends its rank to the SCAN method
    MPI_Scan(&my_rank, &result, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);

    printf("The Scanned result in Process %d is %d.\n", my_rank, result);

    MPI_Finalize();
    return 0;
}
```

Benchmarking Parallel Performance

- Benchmarking parallel program performance **measure how well parallel programs perform against their sequential counterparts** in the "middle area" between reading the dataset and writing the results



Typically, we are going to **ignore** the time spent *initiating MPI processes, establishing communications sockets* between them, and *performing I/O* on sequential devices

Benchmarking Parallel Performance

- MPI provides a function called **MPI_Wtime** that returns the *number of seconds that have elapsed* since some point of time in the past
- Function **MPI_Wtick** returns the precision of the result returned by **MPI_Wtime**

double MPI_Wtime (void)

double MPI_Wtick (void)

- We can benchmark a section of code by putting a pair of calls to function **MPI_Wtime** *before and after the section*. The difference between the two values returned by the function is the number of seconds elapsed

```
double start_time, end_time;
.....
MPI_Init(&argc, &argv);
start_time = MPI_Wtime();

.....parallel execution code.....

end_time = MPI_Wtime();
printf("Process took %f seconds\n",endtime-starttime);
```

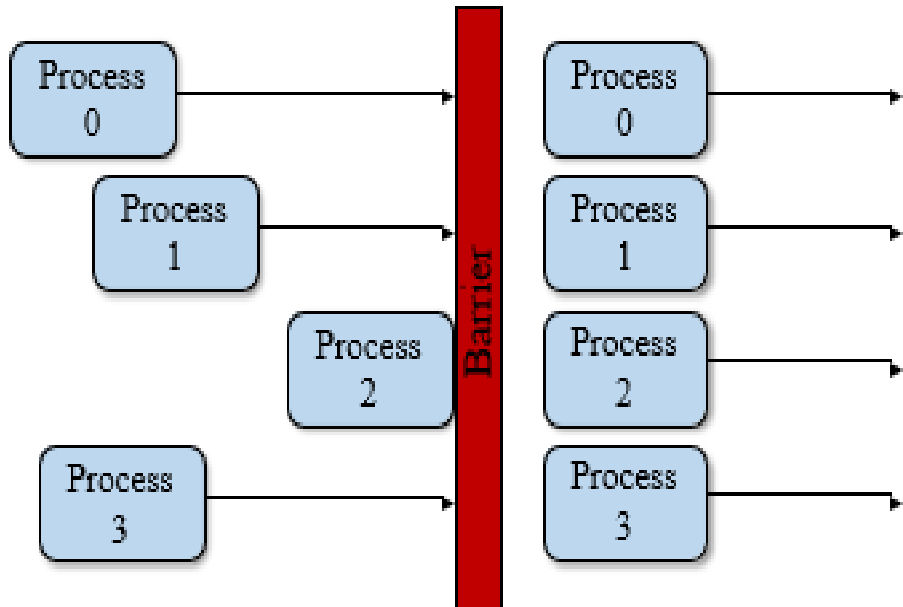
Setting barrier for process synchronization

- From a logical point of view, every MPI process begins execution at the same time, but this is not true in practice
- MPI processes executing on different processors may begin **executing seconds apart**. *This can throw off timings significantly*
- We address this problem by **introducing a barrier synchronization before the first call to MPI_Wtime**.
- No process can proceed beyond a barrier until all processes have reached it
- Hence a barrier ensures that all processes are going into the measured section of code at more or less the same time

Setting barrier for process synchronization

`MPI_Barrier(MPI_COMM_WORLD)`

- Process synchronization (blocking)
 - All processes are forced to wait for each other
 - Will reduce parallelism
- Use only where necessary



```
double start_time, end_time;  
.....  
MPI_Init(&argc, &argv);  
MPI_Barrier(MPI_COMM_WORLD)  
  
start_time = MPI_Wtime();  
  
.....parallel execution code.....  
  
end_time = MPI_Wtime();  
printf("Process took %f seconds\n", endtime-starttime);
```

MPI Error Handling Functions

- When an error is occurred while executing a MPI program typically, the program aborts
- MPI calls a *default error handler* **MPI_ERRORS_ARE_FATAL** every time an MPI error is detected within the communicator
- **MPI_ERRORS_ARE_FATAL** *abort the whole parallel program* as soon as any MPI error is detected
- There is another predefined error handler **MPI_ERRORS_RETURN** which is used to return the generated error for custom handling
- The default error handler **MPI_ERRORS_ARE_FATAL** can be replaced with **MPI_ERRORS_RETURN** by calling function **MPI_Errhandler_set ()**

```
MPI_Errhandler_set(MPI_COMM_WORLD, MPI_ERRORS_RETURN)
```

MPI Error Handling Functions

- Once we've called **MPI_Errhandler_set ()** in our MPI code, the program will no longer abort on having detected an MPI error, instead the error will be returned and we will have to handle it
- MPI standard defines ***error classes***. *Every error code, must belong to some error class*, and the error class for a given error code can be obtained by calling function **MPI_Error_class ()**

- Error code can be converted to comprehensible error messages by calling function **MPI_Error_string ()**

```
MPI_Error_string(int errorcode, char *string, int *resultlen)
```


MPI Error Handling Functions

```
#include "mpi.h"

#include <stdio.h>

void ErrorHandler(int error_code);

int main(int argc, char *argv[]){
    int C=3;

    int size, my_rank, len, error_code;

    MPI_Init(&argc, &argv);

    MPI_Errhandler_set(MPI_COMM_WORLD, MPI_ERRORS_RETURN);

    error_code = MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    error_code = MPI_Comm_size(MPI_COMM_WORLD, &size);

    //error_code = MPI_Comm_size(C, &size);

    ErrorHandler(error_code);

    printf ("Number of processes= %d My rank= %d \n", size, my_rank);

    MPI_Finalize();
}
```

```
void ErrorHandler(int error_code){
    if (error_code != MPI_SUCCESS){
        char error_string[BUFSIZ];
        int length_of_error_string, error_class;
        MPI_Error_class(error_code, &error_class);
        MPI_Error_string(error_code, error_string, &length_of_error_string);
        printf("%d %s\n", error_class, error_string);
    }
}
```

MPI Documentation

MPI Reference